

PRAKTIKUM SISTEM OPERASI

MODUL 11
(MEMORY XINU)



Oleh:

(Muhammad Daniel Anugrah Pratama) (2311104063)

PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM

2026

JURNAL/TP

1. Buatlah perintah baru bernama freememory yang memiliki dua fungsi berikut:
 - a. Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Code :

```
#include <xinu/h>

shellcmd xsh_freememory(int nargs, char *args[]) {
    struct memblk *cur;
    uint32 TotalFreespace = 0;

    kprintf("Address\t\tSize\n");
    kprintf("-----\n");

    cur = memlist.mnext;

    while(cur != null) {
        kprintf("0x%08\t\t%d\n",
            (uint32)cur,
            cur->mlength);

        TotalFreespace += cur->mlength;

        cur = cur->mnext;
    }

    return OK;
}
```

C Tab Width: 8 Ln 28, Col 2 INS

```
Welcome to Xinu!

xsh $ freememory
Address      Length(dec)  Length(hex)
-----
0x0018E110   -975128      0x005F7FC4
0x00100000    5076848      0x005F7FC4
0x005E8000     57344        0x005F7FC4

Total Free Memory = 4153064 bytes
xsh $
```

CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Online 0:0 | ttyS8

2. Jawablah pertanyaan berikut:

- a. Mengapa Xinu memisahkan data segment dan BSS segment?
- b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
- c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
- d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
- e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban :

- a. Data Segment digunakan untuk menyimpan variabel global yang sudah memiliki nilai awal, sedangkan BSS Segment digunakan untuk variabel global yang belum diinisialisasi. Pemisahan ini membuat penggunaan memori lebih efisien karena variabel pada BSS tidak perlu disimpan nilainya di file executable dan cukup diinisialisasi menjadi nol saat sistem dijalankan.
- b. Saat proses baru dibuat atau heap dialokasikan, sebagian free space akan digunakan sehingga ukuran free space berkurang. Sebaliknya, ketika stack atau heap dilepaskan kembali ke sistem, free space akan bertambah. Oleh karena itu ukuran free space berubah secara dinamis selama eksekusi program berlangsung.
- c. Heap lebih berpotensi menimbulkan masalah karena dealokasinya tidak dilakukan secara otomatis ketika proses berakhir. Programmer harus membebaskan memori secara manual menggunakan fungsi yang sesuai. Selain itu heap dapat mengalami fragmentasi dan permintaan memori dapat melebihi free space yang tersedia. Sebaliknya, stack dikelola otomatis saat fungsi atau proses dibuat dan diakhiri.
- d. Xinu menggunakan linked list karena jumlah dan ukuran free block selalu berubah akibat proses alokasi dan dealokasi memori. Linked list memudahkan sistem untuk menambah, menghapus, dan menelusuri blok memori kosong tanpa harus memindahkan data lain di memori. Setiap node menyimpan alamat blok kosong, ukuran blok, dan pointer ke blok kosong berikutnya.
- e. Tantangan utama penggunaan heap adalah pengelolaan memori yang harus dilakukan secara manual oleh proses yang mengalokasikannya. Jika memori tidak dibebaskan, dapat terjadi memory leak. Selain itu heap juga rentan terhadap fragmentasi dan kegagalan alokasi ketika free space yang tersedia tidak cukup untuk memenuhi permintaan memori.